

Project Proposal: File Encryption

Blockchain Project: Milestone 3

Reva Laxmi Chauhan

1 INTRODUCTION

The Patient Health Records System is an innovative and secure decentralized platform aimed at storing and managing patient health records in a tamper-proof and accessible manner for patients. Traditional methods relying on centralized systems like Excel sheets will be replaced by a system using blockchain technology, ensuring data integrity and accessibility.

2 UNDERSTANDING THE PROJECT

My project is a blockchain-based web application integrating a Node.js/Express backend with a frontend interface, focused on securely managing patient records using Hyperledger Fabric.

It features smart contracts for secure transactions, user-friendly APIs for registration and data access, and security measures including password hashing and authentication.

The frontend, likely built with web technologies like HTML, CSS, and JavaScript, provides an intuitive user interface for interacting with the blockchain backend. This setup ensures a secure, decentralized, and efficient management of sensitive healthcare data.

3 WHY ARE WE USING BLOCKCHAIN AND HYPERLEDGER?

Because we want an immutable system, especially in this case. Where we are relying on access controls. And also, because we want transparency throughout this system.

Hyperledger is like a secure system for storing patient health records. Only authorized people can access or change the information, making sure it stays private and tamper-proof ie immutable in this case from one side (or maybe all depending on how we proceed). The code uses Hyperledger's security to protect sensitive patient data and control who can see or modify it.

4 SYSTEM FEATURES

Patient Login: Patients can securely log in and input their details: Name, Age, Contact Information, Gender, Doctor's Name, and Specific Illness. These details will be stored on the ledger

4.1 EXPLANATION

4.1.1 THE FOLLOWING IS WHAT I FINALLY IMPLEMENTED IN MY CODE. Firstly, this is the GitHub link where you can see the initial code of milestone 1 and 2.

<https://github.com/RevaLaxmi/Blockchain-CS2361.git>

New github link to the final project:

<https://github.com/RevaLaxmi/Blockchain-Patient-Record-System.git>

(please let me know if files have to be uploaded in another manner - i have uploaded files manually)

Old code explained

These are the things included in the "fabchat" folder.

Sure, let's break down this code into simpler terms:

1. **initLedger Function:** - This is like the setup function. When the system starts, it just prints messages saying that the Patient-Doctor Management System is ready to go.

2. **registerPatient Function:** - This function is used to add a new patient to the system. - It takes the patient's name, contact info, age, gender, and illness details. - The function splits the name and contact info, then creates a unique ID for the patient. - It also creates a secret key (like a special password) for each patient. - The patient's data is then locked using this key (encrypted) and stored.

3. **generateUniqueOddId and generateUniqueEvenId Functions:** - These functions create unique IDs for patients and doctors. - For patients, the IDs are odd numbers, and for doctors, they are even numbers. - The system checks to make sure no one else has the same ID.

4. **registerDoctor Function:** - This function adds a new doctor to the system. - It takes the doctor's name and contact info and creates a unique ID for them. - Then it saves the doctor's details in the system.

5. idExists Function: - This function checks if an ID already exists in the system.
- It helps prevent two people from having the same ID.

6. getPatientDataBasedOnRequester Function: - This function lets someone get a patient's information. - It checks if the person asking for the info is the patient themselves or their doctor. - If it's the patient or their doctor, the function unlocks (decrypts) the patient's data and shows it. - If not, it doesn't allow access to the data.

7. decryptPatientData Function: - This function is specifically for unlocking a patient's data. - It takes the patient's ID, finds their data, and then uses the secret key to unlock it.

In summary, this code is about managing patient and doctor data in a secure way. It adds patients and doctors to the system, locks patient data for security, and only lets the right people (like the patient or their doctor) see that data. The use of unique IDs and secret keys ensures that all information is kept safe and private.

So the only difference is the implementation of a pdf file/txt versus a patient entering details. Since I am running short on time, i will not pursue this change and I will continue with what I have. But I hope the difference has been made clear.

4.1.2 MILESTONE 3 CODE IMPLEMENTATION: NEW CODE EXPLAINED We have the following according to our fabchat folder and invoke.js:

1. Module and Class Dependencies 'fabric-contract-api': This module provides APIs for developing smart contracts. 'fabric-shim': This is used to interact with the blockchain ledger. The 'ClientIdentity' class from this module is useful for identifying the participants (e.g., patients) interacting with the contract. 'crypto': A core Node.js module for cryptographic functionality. It's used here for hashing passwords.

2. Contract Class Definition 'FabChat' extends 'Contract', making it a smart contract that can be deployed on a Hyperledger Fabric network.

3. initLedger Function This is a standard initialization method for the ledger. It doesn't do much in this implementation besides logging an informational message.

4. **registerPatient Function Purpose:** To create a new patient record on the blockchain. Parameters: 'patientName', 'phoneNumber', 'referringDoctor', 'illness', 'medication', 'password'. Process: It hashes the password using SHA-256, a cryptographic hash function, for secure storage. Creates a patient record object with all the provided details, including the hashed password. Stores this record in the ledger using 'ctx.stub.putState()'. The patient's name is used as the key.

5. **accessPatientRecord Function Purpose:** To retrieve a patient's record from the blockchain. Parameters: 'patientName', 'phoneNumber', 'password'. Process: Retrieves the patient record using 'ctx.stub.getState()'. If the record doesn't exist, it throws an error. Hashes the provided password and compares it with the stored hash. If the phone number and password hash match, it returns the patient data; otherwise, it throws an error for unauthorized access.

Cryptography for Security crypto: This Node.js module is used for cryptographic operations, crucial in blockchain for ensuring data integrity and security.

Password Hashing: FabChat uses SHA-256, a cryptographic hash function, to securely hash passwords before storing them on the blockchain. This is important because it prevents the exposure of plaintext passwords, enhancing security.

The use of cryptographic hashing ensures that even if someone accesses the data on the blockchain, they cannot reverse-engineer the original password from its hash.

5 UI FOR THE PROJECT

5.1 EXPLANATION

Each dashboard will have user-specific functionalities and data visibility. Hopefully making it easy for users to access and manage health records while maintaining data privacy and security.

The UI will likely feature a clean and organized layout, offering clear sections for different functions, and ensuring an easy and efficient user experience. It should also incorporate responsive design elements to adapt well to various devices, ensuring accessibility for both desktop and mobile users.

To create a backend for the Records system, we are using Hyperledger Fabric, which provides a secure blockchain infrastructure. It allows for managing patient data in a way that's tamper-proof and ensures privacy and controlled access to sensitive information.

For the front end, we will use React. It is an efficient JavaScript library for building user interfaces. It offers a responsive and interactive interface, allowing patients and doctors to easily interact with the system, input health details, and access patient records.

This combination ensures a secure, tamper-proof backend (Hyperledger Fabric) for storing records, while the React frontend offers a user-friendly interface for easy and seamless interaction with the system.

UPDATE: I have completed the front end and backend (however there are errors):

Backend (Node.js/Express with Hyperledger Fabric)

The backend of my application is built with Node.js and Express, which act as the server-side environment and framework, respectively. This part of the application handles HTTP requests, interacts with the Hyperledger Fabric network, and performs business logic.

1. Node.js: I am running JavaScript on the server side.
2. Express.js: Express is a minimal and flexible Node.js web application framework that provides a robust set of features to develop web and mobile applications.
3. Hyperledger Fabric Integration: Blockchain Network: Hyperledger Fabric is the distributed ledger framework. So we have our fabchat file there - where we have our patient record management system.
Smart Contracts and Chaincode: My backend interacts with the blockchain network, specifically with smart contracts (or chaincode). These are pieces of code that execute on the blockchain nodes and manage the ledger state based on transactions.
4. API Endpoints: My backend exposes various API endpoints (like '/registerUser', '/enrollAdmin', etc.) that the frontend can consume. These endpoints abstract the complexity of blockchain transactions and present a simple interface to the frontend.
5. Security and Authentication: Handling security in the backend, especially when dealing with blockchain, is crucial. This includes managing authentication, authorization, and secure communications.

Frontend

The frontend of my application is the user interface that interacts with my backend server. It's built using HTML, CSS, and JavaScript, and can be enhanced with various frameworks or libraries (like React in this case).

1. User Interface (UI): The UI sends requests to the backend and displays the received data to the users.
2. HTTP Requests: The frontend makes HTTP requests to the backend and handles responses. These requests are triggered by user actions like clicking buttons, submitting forms, etc.